

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri
Ders Projesi

Ad-Soyad: Derda Enes Tuncer

Öğrenci No: 21291136

GitHub: github.com/derdatuncer/sql-projects

Proje 5 Video: www.youtube.com/watch?v=HCkAYQ_Miho

Proje 6 Video: www.youtube.com/watch?v=RV0S1G4Q93s

Proje 7 Video: www.youtube.com/watch?v=HpMqFHUBQNO

Contents

1. Proje 5 – Veritabanı Yükleme Ve ETL Süreçleri	3
1.1 Projenin Amacı ve Kapsamı	3
1.2 Test Ortamının Hazırlanması	3
1.3 Veri Keşfi ve Profilleme	3
1.4 Veri Dönüştürme ve Kurtarma Stratejileri	4
1.4.1 Metin Standartlaştırma ve Mapping Tabloları	4
1.4.2 İstatistiksel Veri Doldurma	4
1.4.3 Çakışma Çözümü ve Boşluk Doldurma	4
1.5 Yükleme ve Karantina	5
1.6 Sonuçlar	5
2. Proje 6 – Veritabanı Yükseltme ve Sürüm Yönetimi	6
2.1 Projenin Amacı ve Kapsamı	6
2.2 Test Ortamının Hazırlanması	6
2.3 Veritabanı Yükseltme Stratejisi ve Uygulaması	7
2.4 Sürüm Yönetimi ve Değişiklik İzleme	8
2.5 Felaket Senaryosu, Test ve Geri Dönüş Planı	8
2.6 Sonuç	10
3. Proje 7 – Veritabanı Yedekleme ve Otomasyon Çalışması	10
3.1 Projenin Amacı ve Kapsamı	10
3.2 Test Ortamının Hazırlanması	10
3.3 Yedekleme Süreci Otomatizasyonu	10
3.4 Otomatik Yedekleme Uyarıları	11
3.5 Yedekleme Raporları	12
3.6 Sonuç	12

1. Proje 5 – Veritabanı Yükleme Ve ETL Süreçleri

1.1 Projenin Amacı ve Kapsamı

Bu proje, kurumsal mimarilerde sıkça karşılaşılan "Kirli Veri" (Dirty Data) problemini çözmek amacıyla geliştirilmiş bir Veri Mühendisliği uygulamasıdır. İçerisinde yapısal bozukluklar, eksik değerler, tutarsız formatlar ve çakışan anahtarlar barındıran ham bir finansal veri seti, SQL Server üzerinde tasarlanan 3 katmanlı bir ETL pipeline'ı ile işlenmiştir.

1.2 Test Ortamının Hazırlanması

Proje gerçekleştirilirken dış kaynaktan kirli bir finansal veri seti kullanılmıştır. CSV formatında bulunan veri seti MSSQL'e veri tabanına aktarılmıştır. Veri seti oldukça kirli olduğu için başarı oranı hedefi düşük tutulmuştur.

1.3 Veri Keşfi ve Profilleme

Sürecin ilk adımında, veritabanına alınan ham kayıtlar üzerinde bir keşif çalışması yapılmış ve veri kalitesi raporu tablosu oluşturulmuştur. Bu aşamada aşağıdaki majör kirlilikler tespit edilmiştir:

- Müşteri ve İşlem ID'lerinde eksiklikler.
- "2025-02-30" (Şubat'ın 30 çekmemesi) veya "2023-13-01" (13. ayın olmaması) gibi irrasyonel ve imkansız tarihler.
- Fiyat kolonunda "\$" gibi metinsel ifadelerin yer alması ve negatif değerler olması.
- Miktar kolonunda negatif ve ondalıklı değerler.
- Ürün tipi kolonunda yazım farklılıkları.

	Detaylı Profilleme Metriği	Tespit Edilen Hata Sayısı
1	1. Toplam Kayıt Sayısı	100000
2	2. Eksik / Bos Transaction_ID	5018
3	3. Eksik / Bos Customer_ID	4878
4	4. Gecersiz Tarih (Örn: 13. Ay / Uyumsuz Format)	68261
5	5. Mantıksız (-) Negatif veya Sıfır Miktar	31619
6	6. İçinde Dolar (\$) veya Harf Olan Bozuk Fiyatlar	6640
7	7. Eksik / Bos Birakılmış Fiyatlar	33497

1.4 Veri Dönüştürme ve Kurtarma Stratejileri

Projenin mühendislik açısından en yoğun katmanı olan Staging aşamasında, verileri çöpe atmak yerine sisteme kazandırmak için üç farklı yöntem uygulanmıştır:

1.4.1 Metin Standartlaştırma ve Mapping Tabloları

Ürün isimlerinde karşılaşılan aşırı kısaltmalar ("Smar", "Tab", "La", "Cof") ve ödeme yöntemlerindeki karmaşa ("creditcard", "pay pal") veritabanında dinamik olarak oluşturulan Referans (Mapping) tabloları ile çözülmüştür. Metinler eşleştirilerek "Smartphone", "Laptop", "Credit Card" gibi kurumsal formlara dönüştürülmüştür. İşlem durumları (Status) da bu yöntemle "Completed", "Pending", "Failed" formatına çekilmiştir. Statüsü eşleşmeyen veya anlamsız olan işlemler doğrudan reddedilmek üzere NULL bırakılmıştır.

1.4.2 İstatistiksel Veri Doldurma

Veri setindeki imkansız tarihlere sahip kayıtları salt bu nedenden ötürü silmek, istatistiksel analizde sapmalara (bias) yol açacaktır. Bu sebeple en çok tekrar eden değer yöntemi kullanılmıştır. CROSS JOIN mimarisi ile veri setinde en çok işlem yapılan tarih tespit edilmiş ve hatalı tarihe sahip tüm kayıtlar bu istatistiksel gerçeklikle doldurularak kurtarılmıştır. Burada gerçekleştirilen bu yöntem, gerçek uygulamalarda mantıklı bulunmayabilir. Fakat, bu projede bu yöntemi de göstermek üzere kullanılmıştır.

1.4.3 Çakışma Çözümü ve Boşluk Doldurma

Projedeki en kritik mühendislik problemlerinden biri, eksik bırakılan veya aynı numarayla sisteme giren (Duplicate) Transaction_ID sorunudur. Bu sorun, SQL tekniklerinden biri olan "Sanal Sayı Üretici" ile çözülmüştür:

1. Bellekte 200.000 kapasiteli ardışık bir sayı havuzu oluşturulmuştur.
2. Mevcut ve geçerli olan numaralar (EXCEPT komutu ile) bu havuzdan çıkarılarak, tamamen "kullanıma hazır / boşta olan" bir ID listesi elde edilmiştir.
3. Sisteme ID'siz giren veya var olan bir ID ile çakışan (Yinelenen) kayıtlara, bu boşluktaki sayılar sırayla atanmıştır.

Bu sayede Primary Key çakışmaları (Violation of PRIMARY KEY constraint) önlenmiş ve hiçbir veri kaybı yaşanmamıştır.

Bu yöntemler sonucunda oluşturulan staging tablosu aşağıdaki gibidir:

	Row_ID	Transaction_ID	Transaction_Date	Customer_ID	Product_Name	Quantity	Price	Payment_Method	Transaction_Status
1	7	T00007	2023-10-16	C1313	Laptop	10	NULL	Credit Card	Completed
2	1	T0001	2024-08-02	C2205	Headphones	5	420.21	PayPal	NULL
3	2	T0002	2020-02-10	C3156	Coffee Machine	469	445.34	Credit Card	Pending
4	91	T00027	2023-05-16	C308	Coffee Machine	824	577.56	Credit Card	Failed
5	3	T0003	2023-10-16	C2919	Tablet	4	810.99	Credit Card	Completed
6	4	T0004	2020-08-17	C3009	Tablet	7	868.61	PayPal	Pending
7	5	T0005	2023-10-16	C3488	Coffee Machine	10	763.12	PayPal	Completed
8	6	T0006	2021-10-26	C4241	Smartphone	598	NULL	PayPal	Completed
9	8	T0008	2023-10-16	C4736	Headphones	669	86.92	Cash	NULL
10	9	T0009	2023-10-16	C3387	Tablet	10	461.70	PayPal	NULL
11	106	T00091	2023-10-16	C4261	Laptop	670	197.03	PayPal	Failed
12	10	T0010	2023-10-16	C2846	Laptop	1	404.89	Credit Card	Pending
13	125	T00106	2021-12-02	C086	Coffee Machine	667	681.87	PayPal	Pending
14	11	T0011	2023-10-16	C1004	Laptop	10	600.84	PayPal	Completed
15	12	T0012	2023-10-16	C2912	Tablet	608	NULL	PayPal	Completed
16	141	T00125	2023-10-16	C869	Headphones	564	499.88	Credit Card	Failed
17	13	T0013	2020-03-18	C1503	Smartphone	564	NULL	Credit Card	Completed

1.5 Yükleme ve Karantina

Staging katmanında temizlenen ve düzenlenen veriler, doğrudan nihai ambara alınmamış; katı bir kural motorundan geçirilmiştir.

- **Karantina Katmanı (rejected_financial_transactions):** Miktarı veya fiyatı 0 (sıfır) ya da NULL olan, ayrıca işlem durumu saptanamayan kayıtlar; analitik sonuçları bozmamaları adına bu izole tabloya yönlendirilmiştir. (Burdaki mantık kirli verilerin çöpe atılmaması ve daha sonra kullanılmak amacıyla saklanması)
- **Temiz Veri Ambarı (clean_financial_transactions):** Miktarın > 0 ve Fiyatın >= 0 olduğu doğrulanmış kayıtlar bu tabloya yazılmıştır. Tablo seviyesinde PRIMARY KEY ve CHECK kısıtlamaları kurularak, gelecekte manuel olarak eklenebilecek kirli verilerin de sisteme girişi engellenmiştir.

1.6 Sonuçlar

Kurulan bu otomatize ETL pipeline'ı sonucunda, tamamen bozuk ve kullanılamaz halde olan ham veri seti, yapay veri kurtarma algoritmalarının da yardımıyla temizlenmiş ve analiz katmanına aktarılmıştır. Karantinaya ayrılan fireler sadece iş kuralı ihlali (sıfır fiyat/miktar) barındıran, kurtarılması mantıksal olarak imkansız olan kayıtlardan oluşmaktadır.

	Sisteme Giren Ham Veri	Temizlenen ve Kurtarılan Veri	Karantinaya Alınan (Cop)	Basari Orani Yuzdesi
1	100000	52683	47317	52.68

	Transaction_ID	Transaction_Date	Customer_ID	Product_Name	Quantity	Price	Payment_Method	Transaction_Status
1	T0002	2020-02-10	C3156	Coffee Machine	469	445.34	Credit Card	Pending
2	T00027	2023-05-16	C308	Coffee Machine	824	577.56	Credit Card	Failed
3	T0003	2023-10-16	C2919	Tablet	4	810.99	Credit Card	Completed
4	T0004	2020-08-17	C3009	Tablet	7	868.61	PayPal	Pending
5	T0005	2023-10-16	C3488	Coffee Machine	10	763.12	PayPal	Completed
6	T00091	2023-10-16	C4261	Laptop	670	197.03	PayPal	Failed
7	T0010	2023-10-16	C2846	Laptop	1	404.89	Credit Card	Pending
8	T00106	2021-12-02	C086	Coffee Machine	667	681.87	PayPal	Pending
9	T0011	2023-10-16	C1004	Laptop	10	600.84	PayPal	Completed
10	T00125	2023-10-16	C869	Headphones	564	499.88	Credit Card	Failed
11	T0014	2020-06-19	C2992	Coffee Machine	696	905.51	Credit Card	Failed

2. Proje 6 – Veritabanı Yükseltme ve Sürüm Yönetimi

2.1 Projenin Amacı ve Kapsamı

Kurumsal düzeydeki finansal veritabanı sistemleri, artan işlem hacimleri ve yeni güvenlik gereksinimleri doğrultusunda periyodik olarak daha güncel veritabanı motorlarına yükseltilmek zorundadır. Aynı zamanda, canlı sistem üzerinde çalışan tabloların ve prosedürlerin yapısal bütünlüğünün korunması, veri kayıplarının önlenmesi için kritik bir mühendislik gereksinimidir.

Bu projenin amacı; mevcut veritabanı altyapısının sıfır veri kaybı garantisiyle yeni bir sürüme taşınması, veritabanı şemasında yapılan tüm yapısal değişikliklerin otomatik olarak izlenmesi ve olası felaket senaryolarında sistemin saniyeler içinde eski kararlı haline döndürülmesini sağlayan bir veritabanı yönetim mimarisi tasarlamaktır.

2.2 Test Ortamının Hazırlanması

Geliştirilen yükseltme, sürüm kontrolü ve felaket kurtarma senaryolarının izole bir şekilde test edilebilmesi için SQL Server Management Studio (SSMS) üzerinde daha önce kullanılan TransactionDB adlı test veritabanı ortamı ayağa kaldırılmıştır.

2.3 Veritabanı Yükseltme Stratejisi ve Uygulaması

Mevcut TransactionDB veritabanının daha güncel bir SQL Server sürümüne taşınması için “Yan Yana Yükseltme” stratejisi belirlenmiş ve uygulanmıştır. Kesinti süresini minimumda tutmak için süreç üç adımda kurgulanmıştır:

1. **Dışa Aktarma:** Canlı sistemin anlık durumu FULL BACKUP komutuyla güvenli bir .bak dosyası olarak yedeklenmiştir.
2. **Taşıma:** Alınan bu yedek, yeni ortamda RESTORE işlemiyle ayağa kaldırılarak veri aktarımı sağlanmıştır.
3. **Motor Yükseltmesi:** Veritabanı yeni sunucuya aktarıldıktan sonra, mimarının güncel donanım ve yazılım özelliklerinden tam verim alabilmesi için COMPATIBILITY_LEVEL komutu kullanılmış ve veritabanı motoru T-SQL üzerinden SQL Server 2022 seviyesine başarıyla yükseltilmiştir.

```
1  USE master;
2  GO
3
4  -- 1. ESKİ SÜRÜMDEN DIŞA AKTARMA (BACKUP)
5  BACKUP DATABASE TransactionDB
6  TO DISK = 'TransactionDB_EskiSurum.bak'
7  WITH INIT, NAME = 'Yükseltme Öncesi Ana Yedek';
8  GO
9
10 -- 2. YENİ SİSTEME TAŞIMA (RESTORE)
11 ALTER DATABASE TransactionDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
12 RESTORE DATABASE TransactionDB
13 FROM DISK = 'TransactionDB_EskiSurum.bak'
14 WITH REPLACE;
15 ALTER DATABASE TransactionDB SET MULTI_USER;
16 GO
17
18 -- 3. MOTOR YÜKSELTMESİ
19 ALTER DATABASE TransactionDB SET COMPATIBILITY_LEVEL = 160;
20 GO
21
22 -- 4. YÜKSELTME ONAY RAPORU
23 SELECT
24     name AS [Veritabanı_Adi],
25     compatibility_level AS [Güncel_Surum_Kodu],
26     'SQL Server 2022' AS [Hedeflenen_Surum],
27     'Yükseltme Stratejisi Başarıyla Uygulandı!' AS [Durum]
28 FROM sys.databases
29 WHERE name = 'TransactionDB';
30 GO
```

91 % Sorun bulunamadı

Sonuçlar İletiler

	Veritabanı_Adi	Güncel_Surum_Kodu	Hedeflenen_Surum	Durum
1	TransactionDB	160	SQL Server 2022	Yükseltme Stratejisi Başarıyla Uygulandı!

2.4 Sürüm Yönetimi ve Değişiklik İzleme

Veritabanı canlı ortama alındıktan sonra, içerisindeki nesnelerde (tablolar, kısıtlamalar vb.) yapılabilecek izinsiz veya hatalı yapısal değişiklikleri kayıt altına almak için DDL Triggers kullanılarak bir sürüm takip mekanizması kurulmuştur.

Veritabanı seviyesinde (ON DATABASE) çalışan bu tetikleyici mekanizma, sistemdeki her CREATE, ALTER ve DROP komutunu anlık olarak yakalamaktadır. EVENTDATA() fonksiyonu ile; işlemi yapan kullanıcı, işlemin tarihi, hedeflenen nesne ve çalıştırılan SQL komutunun tam metni SchemaChangeLog tablosuna yazılmaktadır. Sisteme test amaçlı eklenen ve silinen bir tablo üzerinden bu mimarinin kusursuz çalıştığı doğrulanmıştır.

	Tarih	Kullanıcı	Olay_Tipi	Nesne	Çalıştırılan_Kod
1	2026-06-05 06:37:32.250	DESKTOP-PF98FUG\Derda	ALTER_TABLE	Test_Tablosu_V1	ALTER TABLE Test_Tablosu_V1 ADD Kolon VARCHAR(10)
2	2026-06-05 06:37:32.250	DESKTOP-PF98FUG\Derda	DROP_TABLE	Test_Tablosu_V1	DROP TABLE Test_Tablosu_V1
3	2026-06-05 06:37:32.247	DESKTOP-PF98FUG\Derda	CREATE_TABLE	Test_Tablosu_V1	CREATE TABLE Test_Tablosu_V1 (ID INT)

2.5 Felaket Senaryosu, Test ve Geri Dönüş Planı

Sistemde yaşanabilecek "mantıksal veri bozulmalarına" (örneğin kritik bir tablonun yanlışlıkla silinmesi) karşı katı bir Geri Dönüş planı tasarlanmış ve kronolojik bir felaket senaryosu ile test edilmiştir:

- Hazırlık ve Güvenlik Noktası:** Sistemde Kritik_Tablo adında, içerisinde hassas veriler barındıran bir yapı oluşturulmuş ve bu sağlıklı anın FULL BACKUP yedeği alınmıştır.
- Kaza Anı:** Veritabanındaki bu kritik tablo, bir hata sonucu DROP komutu ile silinmiştir.
- Tespit:** Sürüm kontrol logları sorgulanarak, tablonun tam silinme saati ve işlemi yapan sorumlu anında tespit edilmiştir.
- İzolasyon ve Geri Yükleme:** Veritabanı hızla SINGLE_USER moduna alınarak dış trafiğe kapatılmış, ardından alınan güvenli yedek üzerinden RESTORE işlemi gerçekleştirilmiştir.
- Doğrulama:** Sistemin tekrar MULTI_USER moduna alınmasının ardından yapılan kontrollerde, silinen tablonun ve içerisindeki kritik verilerin eksiksiz bir şekilde geri döndüğü kanıtlanmıştır.

Aşağıda bu aşamanın sorguları ve çıktıları verilmiştir:


```

USE TransactionDB;
GO

/* =====
0. HAZIRLIK: SİSTEMDE VAR OLAN KRİTİK BİR TABLO
===== */
IF OBJECT_ID('Kritik_Tablo', 'U') IS NOT NULL DROP TABLE Kritik_Tablo;
CREATE TABLE Kritik_Tablo (ID INT, Veri VARCHAR(50));
INSERT INTO Kritik_Tablo VALUES (1, 'Test Verisi');
GO

/* =====
1. GÜVENLİ DURUM YEDEĞİ
===== */
USE master;
GO
BACKUP DATABASE TransactionDB
TO DISK = 'TransactionDB_SafeState.bak'
WITH INIT, NAME = 'Güvenli Durum Yedeği';
GO

/* =====
2. Felaket Senaryosu
===== */
USE TransactionDB;
GO
DROP TABLE Kritik_Tablo;
GO

30 /* =====
31 3. TESPİT AŞAMASI
32 ===== */
33 SELECT TOP 1
34     EventDate AS [Kaza_Saati],
35     DatabaseUser AS [Sorumlu],
36     EventType AS [Hata],
37     ObjectName AS [Silinen_Nesne]
38 FROM SchemaChangeLog
39 WHERE EventType = 'DROP_TABLE'
40 ORDER BY EventDate DESC;
41 GO
42
43 /* =====
44 4. GERİ DÖNÜŞ (ROLLBACK) İŞLEMİ
45 ===== */
46 USE master;
47 GO
48 ALTER DATABASE TransactionDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
49 GO
50 RESTORE DATABASE TransactionDB
51 FROM DISK = 'TransactionDB_SafeState.bak'
52 WITH REPLACE;
53 GO
54 ALTER DATABASE TransactionDB SET MULTI_USER;
55 GO
56
57 /* =====
58 5. Kanıt
59 ===== */
60 -- Juriye verinin kaybolmadığını kanıtlayan son sorgu
61 SELECT * FROM Kritik_Tablo;
62 GO

```

3 % 8 0

B Sonuçlar İletiler

Kaza_Saati	Sorumlu	Hata	Silinen_Nesne
2026-06-05 06:45:28.4...	DESKTOP-PF98FUGİDerda	DROP_TABLE	Kritik_Tablo

ID	Veri
1	Test Verisi

2.6 Sonuç

Geliştirilen bu proje kapsamında, kurumsal bir finansal veritabanının güvenli bir şekilde yükseltilmesi, şema sürüm geçmişinin T-SQL Triggers kullanılarak denetim altında tutulması ve olası felaket anları için sıfır veri kaybı hedefleyen geri dönüş mekanizmaları başarıyla kurgulanmıştır. Ortaya çıkan mimari, sistemin yüksek erişilebilirlik ve felaket kurtarma standartlarını karşılayan bir şekilde simüle edilmiştir.

3. Proje 7 – Veritabanı Yedekleme ve Otomasyon Çalışması

3.1 Projenin Amacı ve Kapsamı

Kurumsal düzeydeki veritabanı sistemleri, içerdikleri verinin kritikliği sebebiyle herhangi bir sistem çökmesi veya donanım arızası durumunda veri kaybı yaşanmamasını garanti altına almak zorundadır. Bu projenin amacı; yedekleme operasyonlarını manuel ve hataya açık insan süreçlerinden arındırarak tam otomatik bir mimariye kavuşturmak, yedekleme geçmişini görsel raporlarla denetlenebilir hale getirmek ve olası görev başarısızlıklarında sistem yöneticilerini anında bilgilendirecek bir uyarı mekanizması inşa etmektir.

3.2 Test Ortamının Hazırlanması

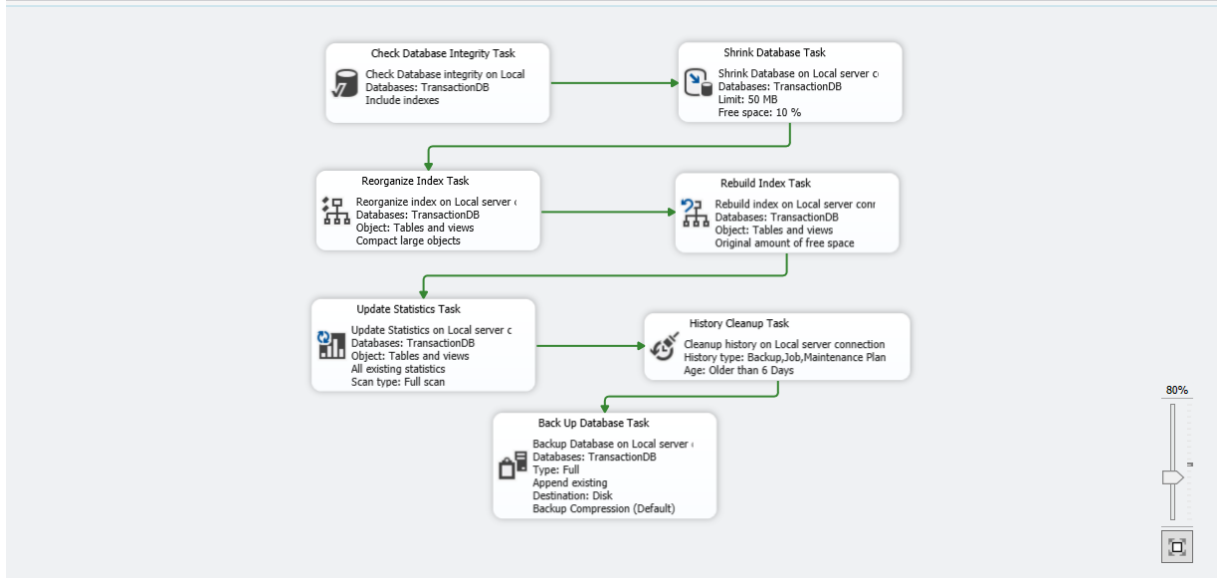
Otomasyon süreçlerinin geliştirilmesi ve test edilebilmesi için SQL Server Management Studio üzerinde arka plan görev yöneticisi olan SQL Server Agent servisi aktif hale getirilmiştir. Sistem e-posta gönderimlerinin ve operatör tanımlamalarının yapılacağı altyapı, msdb sistem veritabanı ile entegre çalışacak şekilde konfigüre edilmiş ve test süreçlerinin izole bir ortamda, görsel araçlar yardımıyla yürütülmesi için gerekli tasarım yetkileri yapılandırılmıştır.

3.3 Yedekleme Süreci Otomatizasyonu

Yedekleme süreçlerinde kurumsal düzeyde bir simülasyon için T-SQL yerine SSMS içerisindeki araçlar kullanılmıştır. Bu bölümde SSMS içerisindeki Maintenance Plan Designer kullanılmıştır. Veritabanının yedekleme sürecinin

her pazar çalışması için özel bir zamanlayıcı tanımlanmıştır ve bakım süreci aşağıda görülebilir.

Subplan	Description	Schedule	Run as
Yedekleme_Planı	Haftalık Yedekleme Planı	Her hafta Pazar günü şu saatte ger...	SQL Server Agent service account



3.4 Otomatik Yedekleme Uyarıları

Sisteme ilk olarak "DBA_Admin" adıyla bir operatör tanımlanmış ve kurumsal e-posta adresi atanmıştır. Ardından, oluşturulan yedekleme görevinin "Alerts" sekmesinde hata yönetimi ayarlanmıştır. Bu sayede; disk dolması, yetki eksikliği veya sunucu kapanması gibi sebeplerle yedekleme görevi tamamlanamazsa, sistemin ilgili operatöre saniyeler içinde "Görev Başarısız Oldu" uyarısı göndermesi garanti altına alınmıştır.

İş Özellikleri - MaintenancePlan.Yedekleme_Planı

Sayfa seçin

- Genel
- Adımlar
- Zamanlamalar
- Uyanlar
- Bildirimler**
- Hedefler

Betik Yardım

İş tamamlandığında gerçekleştirilecek eylemler:

☒ E-posta:	DBA_Admin	İş başarısız olduğunda
☐ Sayfa:		İş başarısız olduğunda
☐ Windows Uygulaması olay günlüğüne yaz:		İş başarısız olduğunda
☐ İş otomatik olarak sil:		İş başarılı olduğunda

3.5 Yedekleme Raporları

SQL Server'ın arka planda yedekleme loglarını tuttuğu msdb.dbo.backupset ve msdb.dbo.backupmediafamily sistem tabloları INNER JOIN mantığıyla birleştirilerek sorgulanmıştır. Geliştirilen bu T-SQL raporu sayesinde; geçmişte alınan tüm yedeklerin türü (Full, Log vb.), tam başlangıç ve bitiş saatleri, MB cinsinden boyutları ve diske yazıldıkları fiziksel dosya yolları detaylı bir şekilde raporlanmıştır.

	Veritabanı	Yedek_Turu	Baslangic_Zamani	Bitis_Zamani	Boyut_MB	Kayitli_Dosya_Yolu	Durum
1	TransactionDB	Full Backup (Tam)	2026-06-05 06:45:28	2026-06-05 06:45:28	46.08	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	BASARILI
2	TransactionDB	Full Backup (Tam)	2026-06-05 06:44:59	2026-06-05 06:44:59	46.08	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	BASARILI
3	TransactionDB	Full Backup (Tam)	2026-06-05 06:44:51	2026-06-05 06:44:51	46.08	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	BASARILI
4	TransactionDB	Full Backup (Tam)	2026-06-05 06:40:57	2026-06-05 06:40:58	46.08	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	BASARILI
5	TransactionDB	Full Backup (Tam)	2026-06-05 06:40:03	2026-06-05 06:40:03	46.09	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	BASARILI
6	TransactionDB	Full Backup (Tam)	2026-06-05 06:29:14	2026-06-05 06:29:14	46.09	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	BASARILI

(Üst kısımdaki raporda bulunan veriler bizim hazırladığımız bakım planı değil önceki projeden kalan backuplardır. Zamanlayıcıya göre bakım planı gerçekleştirildiğinde burada yerini edinecektir.)

Başlangıç Zamanı	Süre (dakika)	Yedekleme Türü	Yedekleme Boyutu	Yedekleme Adı	Cihaz Türü	Kullanıcı Adı	Kurtarma Modeli	Değişiklik Tabanı LSN'si	Son LSN
5.06.2026 06:45:28	0,00	Veritabanı	46,08 MB	Güvenli Durum Yedeği	Disk (geçici)	DESKTOP-PF98FUG\Derda	FULL	Geçerli değil	58000005613600001
5.06.2026 06:44:59	0,00	Veritabanı	46,08 MB	Güvenli Durum Yedeği	Disk (geçici)	DESKTOP-PF98FUG\Derda	FULL	Geçerli değil	58000005586400001
5.06.2026 06:44:51	0,00	Veritabanı	46,08 MB	Güvenli Durum Yedeği	Disk (geçici)	DESKTOP-PF98FUG\Derda	FULL	Geçerli değil	58000005557600001
5.06.2026 06:40:57	0,02	Veritabanı	46,08 MB	Güvenli Durum Yedeği	Disk (geçici)	DESKTOP-PF98FUG\Derda	FULL	Geçerli değil	58000005529600001
5.06.2026 06:40:03	0,00	Veritabanı	46,09 MB	Güvenli Durum Yedeği	Disk (geçici)	DESKTOP-PF98FUG\Derda	FULL	Geçerli değil	58000005502400001
5.06.2026 06:29:14	0,00	Veritabanı	46,09 MB	Yükseltme Öncesi Ana Yedek	Disk (geçici)	DESKTOP-PF98FUG\Derda	FULL	Geçerli değil	58000005335200001

Üstte görülen tablo, SSMS'in hazırladığı standard raporlar kısmından alınmış tablodur.

3.6 Sonuç

Geliştirilen bu proje adımı ile örnek veritabanı altyapısı insan bağımsız bir yedekleme otomasyonuna kavuşturulmuştur. Maintenance Plan Designer kullanılarak oluşturulan esnek iş akışı, T-SQL scripting ile desteklenen derinlemesine sistem raporlaması ve hata anında devreye giren e-posta uyarı sistemleri sayesinde, veritabanı yönetimi gerçekleştirilmiştir.